Thesis

Jason Leff

2026-01-10

Setup

```
rm(list=ls())
data <- read.csv('data_with_pre_data.csv')
set.seed(67)
```

Normalize to Per Minute Data

```
normalize_stats <- function(df) {
  nba_start <- which(names(df) == "NBA_MIN") #split the data
  nba_end   <- which(names(df) == "PRE_G")

  for (i in (nba_start+1):(nba_end-1)) {
    colname <- names(df)[i]
    if (!grepl("PCT", colname, ignore.case = TRUE) && is.numeric(df[[colname]])) { #divide total columns
      df[[colname]] <- round(as.numeric(df[[colname]]) / as.numeric(df[["NBA_MIN"]]),3)
    }
  }
  pre_start <- which(names(df) == "PRE_MP")
  for (i in (pre_start+1):ncol(df)) { #repeat for college
    colname <- names(df)[i]
    if (!grepl("PCT", colname, ignore.case = TRUE) && is.numeric(df[[colname]])) {
      df[[colname]] <- round(as.numeric(df[[colname]]) / as.numeric(df[["PRE_MP"]]),3)
    }
  }
  return(df)
}
per_min_data <- normalize_stats(data)
```

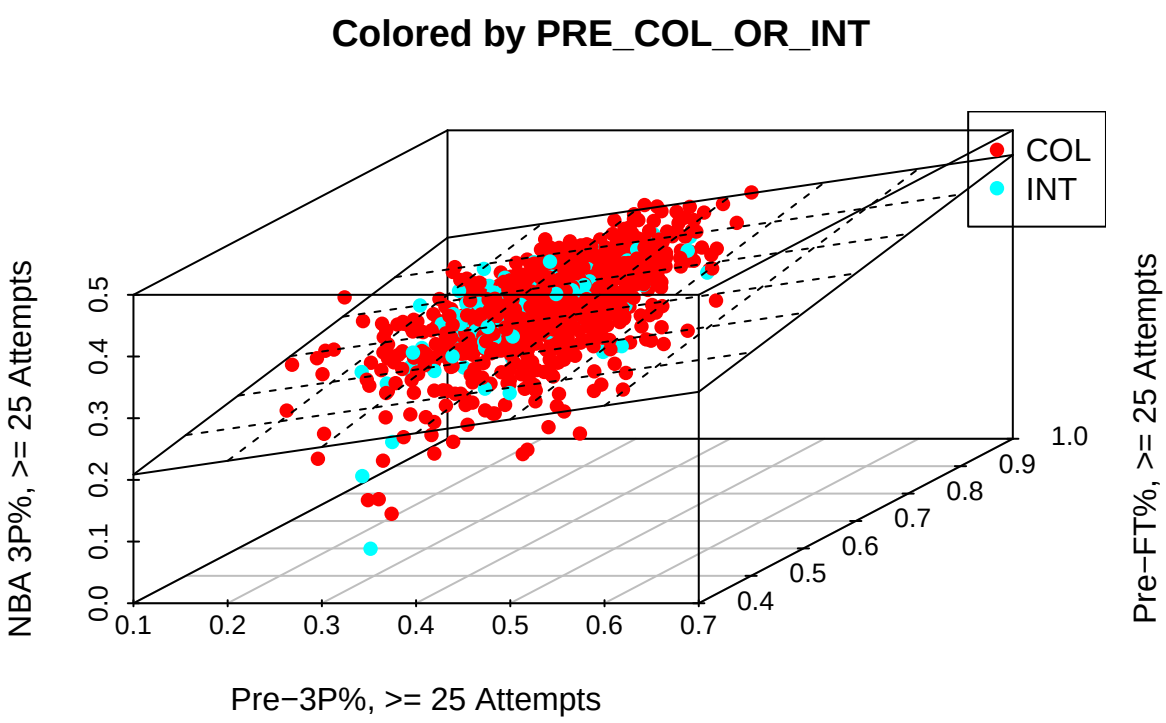
PRE-NBA FT% and 3P% vs NBA 3P% filtered for 25 attempts

```
data_filtered2 <- data %>%
  filter(PRE_3PA >= 25 & PRE_FTA >= 25 & NBA_FG3A >= 25)
```

```r
clean_data <- na.omit(data_filtered2[c("PLAYER","NBA_FG3_PCT", "PRE_3P_PCT", "PRE_FT_PCT", "PRE_COL_or_
model <- lm(NBA_FG3_PCT ~ PRE_3P_PCT + PRE_FT_PCT, data = clean_data)
summary(model)
```

```
##
## Call:
## lm(formula = NBA_FG3_PCT ~ PRE_3P_PCT + PRE_FT_PCT, data = clean_data)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.252344 -0.019491  0.008559  0.033728  0.095787
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.10796    0.01583   6.822 1.49e-11 ***
## PRE_3P_PCT   0.22357    0.03315   6.743 2.51e-11 ***
## PRE_FT_PCT   0.19565    0.02277   8.593  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04865 on 1085 degrees of freedom
## Multiple R-squared:  0.1633, Adjusted R-squared:  0.1618
## F-statistic: 105.9 on 2 and 1085 DF,  p-value: < 2.2e-16
```

```r
color_factor <- as.factor(clean_data$PRE_COL_or_INT)
colors <- rainbow(length(levels(color_factor)))[color_factor]
s3d <- scatterplot3d(clean_data$PRE_3P_PCT, clean_data$PRE_FT_PCT, clean_data$NBA_FG3_PCT,
                     color = colors, pch = 16,
                     xlab = "Pre-3P%, >= 25 Attempts", ylab = "Pre-FT%, >= 25 Attempts", zlab = "NBA 3P
                     main = "Colored by PRE_COL_OR_INT")
s3d$plane3d(model, lty.box = "solid")

legend("topright", legend = levels(color_factor),
       col = rainbow(length(levels(color_factor))), pch = 16)
```



Initialize Train / Test Split

```
per_min_data2 <- filter(per_min_data, NBA_MIN >= 250) # Filter out players with less than 250 min playe
# Specific set of predictors
X <- per_min_data2[, c(8,13,38:40,42,43,45,46,48:50, 52:55, 56:66)]
X[is.na(X)] <- 0
y <- per_min_data2[,20] # per_min_data[,21 or 22] for 3PAPM or 3PCT

df <- data.frame(X, response = y)
df_scaled <- data.frame(scale(df[, -ncol(df)]), response = df$response)

# Split train and test
set.seed(67)
train_index <- createDataPartition(df_scaled$response, p = 0.8, list = FALSE)
train_data <- df_scaled[train_index, ]
test_data  <- df_scaled[-train_index, ]

x_train <- as.matrix(train_data[, -ncol(train_data)])
y_train <- train_data$response
x_test <- as.matrix(test_data[, -ncol(test_data)])
y_test <- test_data$response
```

Initializing Storing Results

```
model_results <- data.frame(
  Model = character(),
  RMSE = numeric(),
  MAE = numeric(),
  R2 = numeric(),
  stringsAsFactors = FALSE
)
```

Function for calculating accuracy metrics

```
calculate_metrics <- function(predictions, actual) {
  rmse_val <- sqrt(mean((predictions - actual)^2))
  mae_val <- mean(abs(predictions - actual))
  r2_val <- 1 - sum((actual - predictions)^2) / sum((actual - mean(actual))^2)
  return(c(rmse_val, mae_val, r2_val))
}
```

Linear Regression

```
lm_model <- lm(response ~ ., data = train_data)
lm_pred <- predict(lm_model, newdata = test_data)
lm_metrics <- calculate_metrics(lm_pred, y_test)

model_results <- rbind(model_results, data.frame(
  Model = "Linear Regression (OLS)",
  RMSE = lm_metrics[1],
  MAE = lm_metrics[2],
  R2 = lm_metrics[3]
))
summary(lm_model)
```

```
##
## Call:
## lm(formula = response ~ ., data = train_data)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.054162 -0.009583 -0.001521  0.009412  0.059895
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)     3.452e-02  4.769e-04  72.377  < 2e-16 ***
## WEIGHT          1.671e-03  9.042e-04   1.848 0.064847 .
## HEIGHT_INCHES  -6.187e-04  9.913e-04  -0.624 0.532714
```

```
## PRE_G           5.670e-03  2.311e-03   2.453 0.014315 *
## PRE_GS          1.824e-03  1.674e-03   1.090 0.275986
## PRE_MP         -8.484e-03  2.986e-03  -2.841 0.004574 **
## PRE_FGA        -1.633e-03  2.369e-03  -0.689 0.490688
## PRE_FG_PCT      5.697e-05  1.569e-03   0.036 0.971047
## PRE_3PA         1.713e-02  2.246e-03   7.629 5.12e-14 ***
## PRE_3P_PCT      1.008e-03  5.745e-04   1.754 0.079745 .
## PRE_2PA         3.625e-04  2.570e-03   0.141 0.887859
## PRE_2P_PCT     -1.378e-03  3.844e-03  -0.358 0.720087
## PRE_eFG_PCT     2.367e-03  3.936e-03   0.601 0.547790
## PRE_FTA        -1.156e-03  1.092e-03  -1.058 0.290303
## PRE_FT_PCT      3.832e-03  7.722e-04   4.963 8.05e-07 ***
## PRE_ORB         3.695e-04  8.755e-04   0.422 0.673120
## PRE_DRB         2.114e-03  1.005e-03   2.104 0.035627 *
## PRE_TRB        -4.512e-04  1.337e-03  -0.338 0.735724
## PRE_AST         3.210e-03  8.390e-04   3.827 0.000137 ***
## PRE_STL        -1.667e-03  7.251e-04  -2.299 0.021680 *
## PRE_BLK        -1.981e-03  7.470e-04  -2.652 0.008114 **
## PRE_TOV        -4.288e-03  6.670e-04  -6.429 1.91e-10 ***
## PRE_PF          1.492e-03  7.551e-04   1.976 0.048432 *
## PRE_PTS         5.921e-04  2.492e-03   0.238 0.812240
## PRE_FG_pct_inc -1.379e-03  8.929e-04  -1.544 0.122885
## PRE_3P_pct_inc  9.314e-04  6.142e-04   1.516 0.129707
## PRE_2P_pct_inc -4.145e-04  8.600e-04  -0.482 0.629936
## PRE_FT_pct_inc  4.103e-04  5.845e-04   0.702 0.482932
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01593 on 1091 degrees of freedom
## Multiple R-squared:  0.6385, Adjusted R-squared:  0.6296
## F-statistic: 71.38 on 27 and 1091 DF,  p-value: < 2.2e-16
```

OLS with CV

```r
set.seed(67)
cv_folds <- createFolds(train_data$response, k = 10)
cv_rmse_ols <- numeric(10)
cv_mae_ols <- numeric(10)
cv_r2_ols <- numeric(10)

for(i in 1:10) {
  train_fold <- train_data[-cv_folds[[i]], ]
  valid_fold <- train_data[cv_folds[[i]], ]

  fold_model <- lm(response ~ ., data = train_fold)
  fold_pred <- predict(fold_model, newdata = valid_fold)

  cv_rmse_ols[i] <- sqrt(mean((fold_pred - valid_fold$response)^2))
  cv_mae_ols[i] <- mean(abs(fold_pred - valid_fold$response))
  cv_r2_ols[i] <- 1 - sum((valid_fold$response - fold_pred)^2) /
    sum((valid_fold$response - mean(valid_fold$response))^2)
```

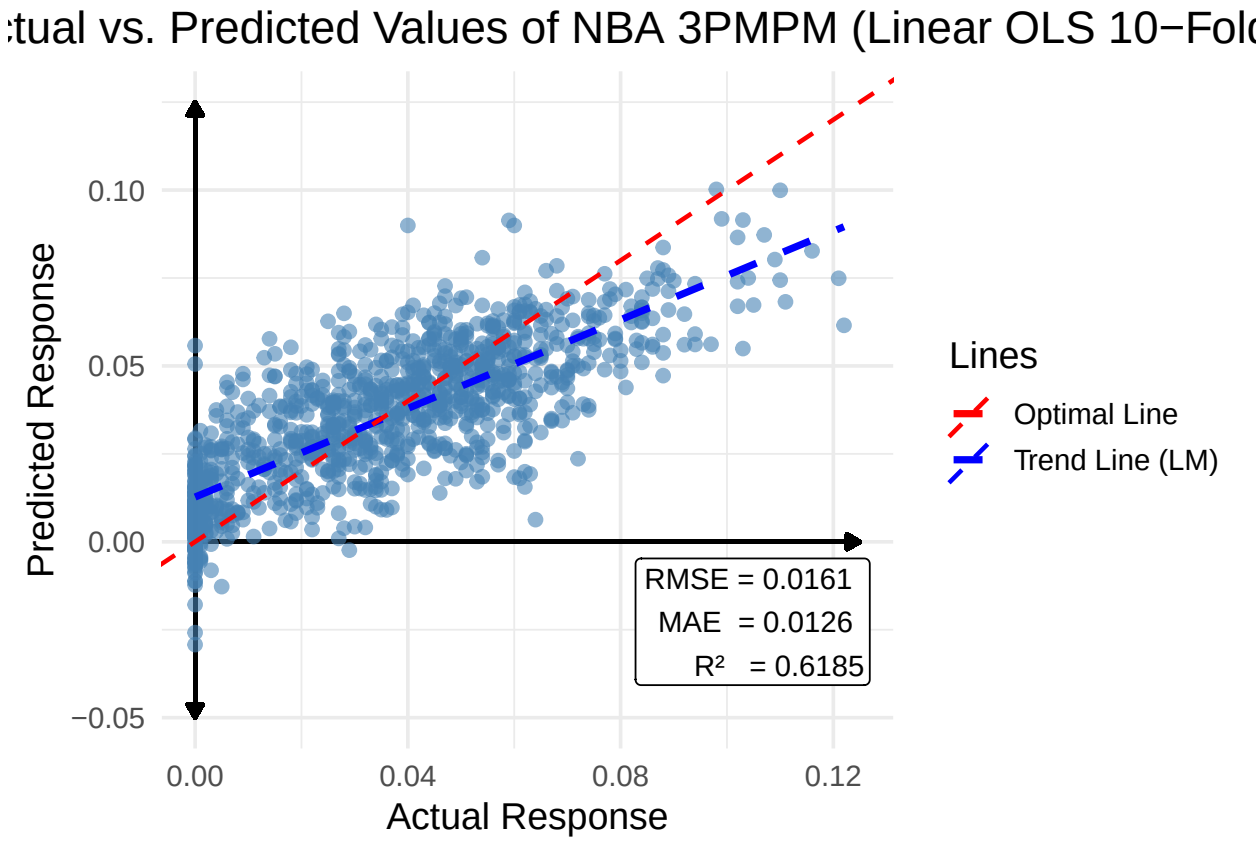
```
}

model_results <- rbind(model_results,
  data.frame(
    Model = "Linear Regression (10-fold CV)",
    RMSE = mean(cv_rmse_ols),
    MAE = mean(cv_mae_ols),
    R2 = mean(cv_r2_ols)
))
```

```
all_actual <- c()
all_predicted <- c()

for(i in 1:10) {
  train_fold <- train_data[-cv_folds[[i]], ]
  valid_fold <- train_data[cv_folds[[i]], ]

  fold_model <- lm(response ~ ., data = train_fold)
  fold_pred <- predict(fold_model, newdata = valid_fold)

  # Store predictions
  all_actual <- c(all_actual, valid_fold$response)
  all_predicted <- c(all_predicted, fold_pred)

  cv_rmse_ols[i] <- sqrt(mean((fold_pred - valid_fold$response)^2))
  cv_mae_ols[i] <- mean(abs(fold_pred - valid_fold$response))
  cv_r2_ols[i] <- 1 - sum((valid_fold$response - fold_pred)^2) /
    sum((valid_fold$response - mean(valid_fold$response))^2)
}

df_plot <- data.frame(
  Actual = all_actual,
  Predicted = all_predicted
)

ggplot(df_plot, aes(x = Actual, y = Predicted)) +

  # --- Custom axes with arrows ---
  geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +
  geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +
  geom_segment(aes(x = 0, xend = 0, y = 0, yend = -0.05),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +

  # --- Points ---
  geom_point(color = "steelblue", alpha = 0.6) +

  # --- Trend line with legend ---
  geom_smooth(aes(color = "Trend Line (LM)"),
```

```
            method = "lm", se = FALSE, linetype = "dashed", show.legend = TRUE) +

  # --- 1:1 line with legend (fixed) ---
  geom_abline(aes(intercept = 0, slope = 1, color = "Optimal Line"),
              linetype = "dashed", linewidth = 0.8, show.legend = TRUE) +

  scale_color_manual(
    name = "Lines",
    values = c(
      "Trend Line (LM)" = "blue",
      "Optimal Line" = "red"
    )
  ) +

  # --- Statistics box ---
  annotate(
    "label",
    x = Inf, y = -Inf,
    hjust = 1.1, vjust = -0.5,
    label = paste(
      "RMSE =", round(mean(cv_rmse_ols), 4), "\n",
      "MAE  =", round(mean(cv_mae_ols), 4), "\n",
      "R²   =", round(mean(cv_r2_ols), 4)
    ),
    size = 4,
    fill = "white"
  ) +

  # --- Labels ---
  labs(
    title = "Actual vs. Predicted Values of NBA 3PMPM (Linear OLS 10-Fold CV)",
    x = "Actual Response",
    y = "Predicted Response"
  ) +

  # --- Theme ---
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.3)
  )
```

```
## Warning in geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0), arrow = arrow(type = "closed", : A
## i Please consider using `annotate()` or provide this layer with data containing
##   a single row.

## Warning in geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125), arrow = arrow(type = "closed", : A
## i Please consider using `annotate()` or provide this layer with data containing
##   a single row.

## Warning in geom_segment(aes(x = 0, xend = 0, y = 0, yend = -0.05), arrow = arrow(type = "closed", : A
## i Please consider using `annotate()` or provide this layer with data containing
##   a single row.
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



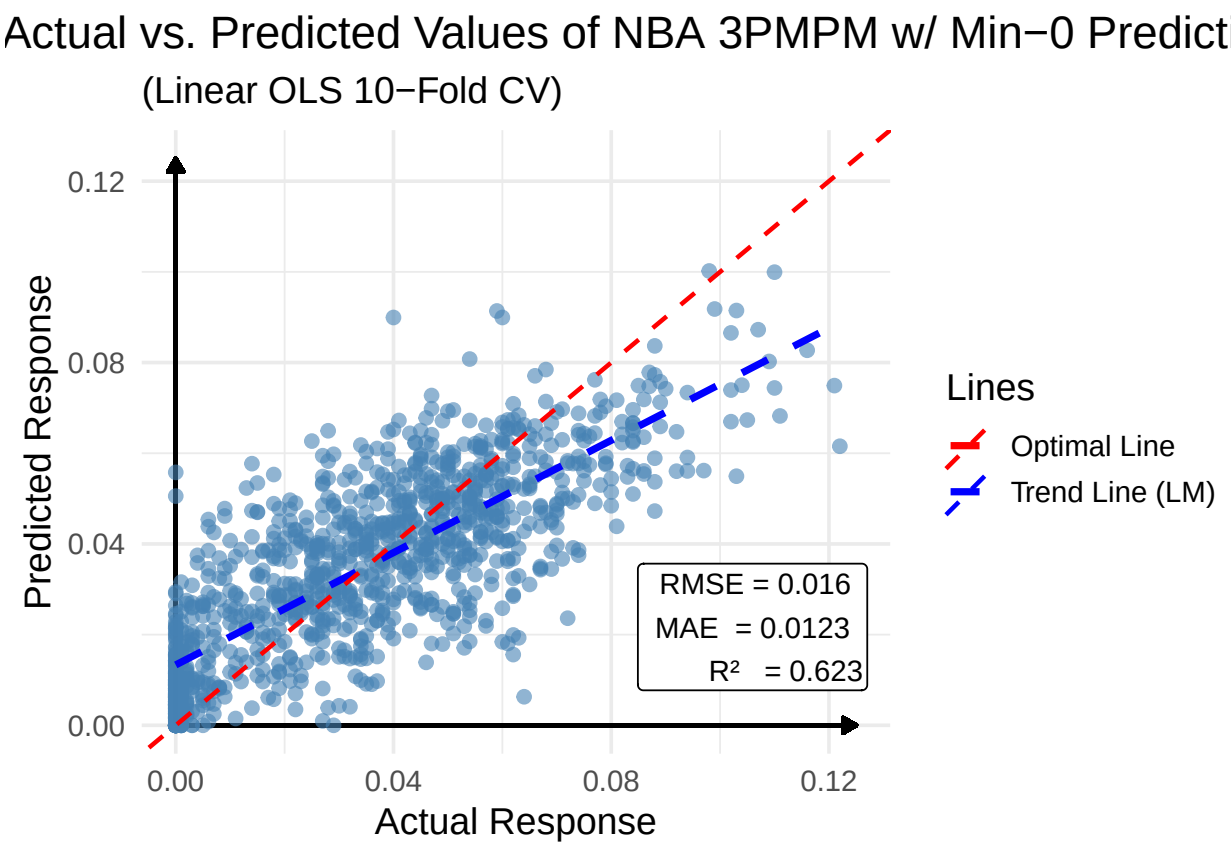
Force Predicted to minimum 0

```
all_actual <- c()
all_predicted <- c()

for(i in 1:10) {
  train_fold <- train_data[-cv_folds[[i]], ]
  valid_fold <- train_data[cv_folds[[i]], ]

  fold_model <- lm(response ~ ., data = train_fold)
  fold_pred <- predict(fold_model, newdata = valid_fold)
  fold_pred <- pmax(fold_pred, 0)

  # Store predictions
  all_actual <- c(all_actual, valid_fold$response)
  all_predicted <- c(all_predicted, fold_pred)

  cv_rmse_ols[i] <- sqrt(mean((fold_pred - valid_fold$response)^2))
  cv_mae_ols[i] <- mean(abs(fold_pred - valid_fold$response))
  cv_r2_ols[i] <- 1 - sum((valid_fold$response - fold_pred)^2) /
    sum((valid_fold$response - mean(valid_fold$response))^2)
}

df_plot <- data.frame(
```

```
  Actual = all_actual,
  Predicted = all_predicted
)

ggplot(df_plot, aes(x = Actual, y = Predicted)) +

  # --- Custom axes with arrows ---
  geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +
  geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +

  # --- Points ---
  geom_point(color = "steelblue", alpha = 0.6) +

  # --- Trend line with legend ---
  geom_smooth(aes(color = "Trend Line (LM)"),
            method = "lm", se = FALSE, linetype = "dashed", show.legend = TRUE) +

  # --- 1:1 line with legend (fixed) ---
  geom_abline(aes(intercept = 0, slope = 1, color = "Optimal Line"),
              linetype = "dashed", linewidth = 0.8, show.legend = TRUE) +

  scale_color_manual(
    name = "Lines",
    values = c(
      "Trend Line (LM)" = "blue",
      "Optimal Line" = "red"
    )
  ) +

  # --- Statistics box ---
  annotate(
    "label",
    x = Inf, y = -Inf,
    hjust = 1.1, vjust = -0.5,
    label = paste(
      "RMSE =", round(mean(cv_rmse_ols), 4), "\n",
      "MAE  =", round(mean(cv_mae_ols), 4), "\n",
      "R²   =", round(mean(cv_r2_ols), 4)
    ),
    size = 4,
    fill = "white"
  ) +

  # --- Labels ---
  labs(
    title = "Actual vs. Predicted Values of NBA 3PMPM w/ Min-0 Predictions",
    subtitle = "(Linear OLS 10-Fold CV)",
    x = "Actual Response",
    y = "Predicted Response"
```

```
  ) +

  # --- Theme ---
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.25)
  )
```

```
## Warning in geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0), arrow = arrow(type = "closed", : A
## i Please consider using `annotate()` or provide this layer with data containing
##   a single row.

## Warning in geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125), arrow = arrow(type = "closed", : A
## i Please consider using `annotate()` or provide this layer with data containing
##   a single row.

## `geom_smooth()` using formula = 'y ~ x'
```




```
model_results <- rbind(model_results,
  data.frame(
    Model = "Linear Regression Min-0 Predictions (10-fold CV)",
    RMSE = mean(cv_rmse_ols),
    MAE = mean(cv_mae_ols),
    R2 = mean(cv_r2_ols)
))
```

```r
# 1. QUADRATIC VERSION OF THE MODEL
# Add quadratic terms to the data
create_linear_quadratic_features <- function(data) {
  response <- data$response
  predictors <- data[, setdiff(names(data), "response"), drop = FALSE]

  # Quadratic terms
  quadratic_terms <- predictors^2
  colnames(quadratic_terms) <- paste0(colnames(predictors), "_sq")

  # Combine
  out <- cbind(predictors, quadratic_terms, response = response)
  return(out)
}

train_data_lq <- create_linear_quadratic_features(train_data)
test_data_lq  <- create_linear_quadratic_features(test_data)

lm_lq <- lm(response ~ ., data = train_data_lq)
lm_lq_pred <- predict(lm_lq, newdata = test_data_lq)
lm_lq_pred <- pmax(lm_lq_pred, 0)
lm_lq_metrics <- calculate_metrics(lm_lq_pred, y_test)

model_results <- rbind(model_results, data.frame(
  Model = "Quadratic OLS",
  RMSE = lm_lq_metrics[1],
  MAE = lm_lq_metrics[2],
  R2 = lm_lq_metrics[3]
))
```

```r
# Initialize storage
all_actual <- c()
all_predicted <- c()

# Determine actual number of folds
n_folds <- length(cv_folds)

cv_rmse_quad <- numeric(n_folds)
cv_mae_quad <- numeric(n_folds)
cv_r2_quad <- numeric(n_folds)

for(i in 1:n_folds) {
  train_fold <- train_data[-cv_folds[[i]], ]
  valid_fold <- train_data[cv_folds[[i]], ]

  # Create quadratic features for each fold
  train_fold_lq <- create_linear_quadratic_features(train_fold)
  valid_fold_lq <- create_linear_quadratic_features(valid_fold)

  # Fit quadratic OLS model
  fold_model <- lm(response ~ ., data = train_fold_lq)
  fold_pred <- predict(fold_model, newdata = valid_fold_lq)
  fold_pred <- pmax(fold_pred, 0)  # Force minimum of zero
```

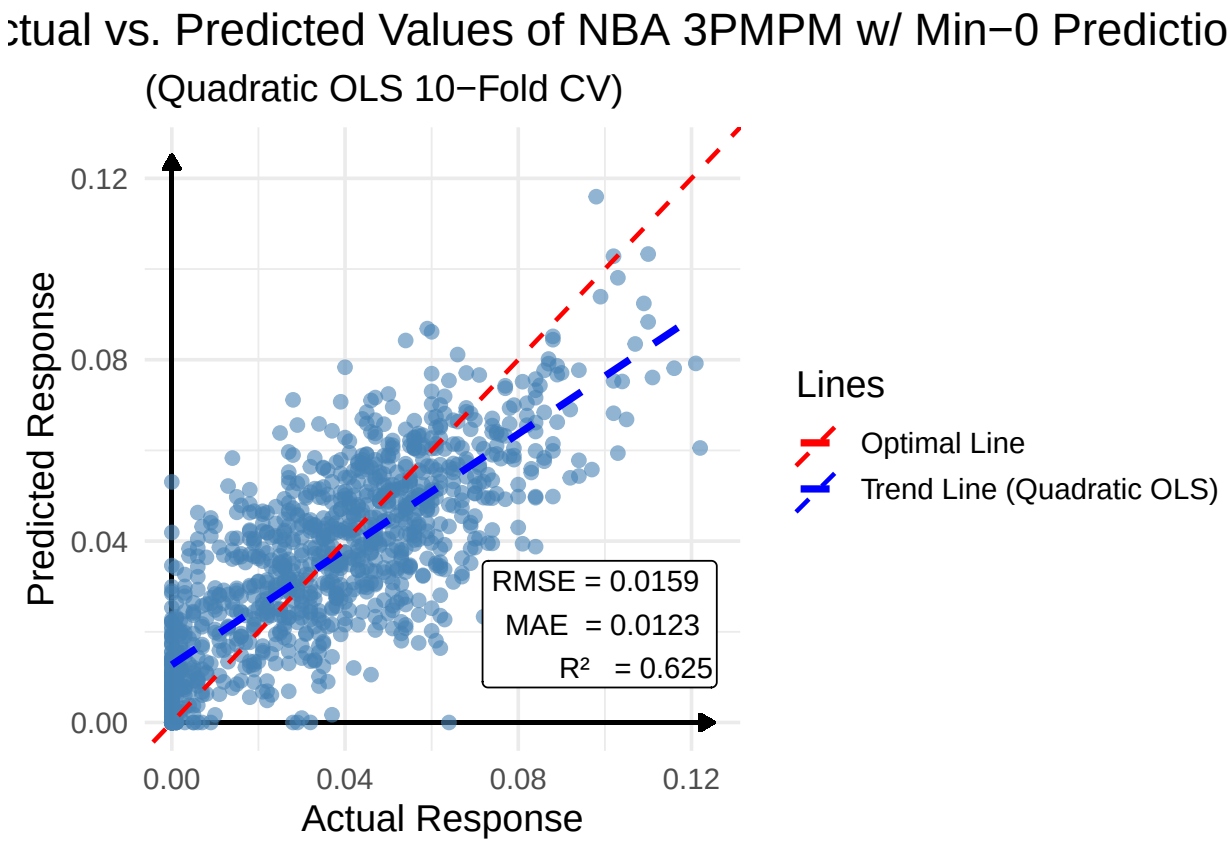
```r
  # Store predictions
  all_actual <- c(all_actual, valid_fold$response)
  all_predicted <- c(all_predicted, fold_pred)

  # Calculate metrics
  cv_rmse_quad[i] <- sqrt(mean((fold_pred - valid_fold$response)^2))
  cv_mae_quad[i] <- mean(abs(fold_pred - valid_fold$response))
  cv_r2_quad[i] <- 1 - sum((valid_fold$response - fold_pred)^2) /
    sum((valid_fold$response - mean(valid_fold$response))^2)
}

# Create plot data
df_plot <- data.frame(
  Actual = all_actual,
  Predicted = all_predicted
)

ggplot(df_plot, aes(x = Actual, y = Predicted)) +

  # --- Custom axes with arrows ---
  geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +
  geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +

  # --- Points ---
  geom_point(color = "steelblue", alpha = 0.6) +

  # --- Trend line with legend ---
  geom_smooth(aes(color = "Trend Line (Quadratic OLS)"),
            method = "lm", se = FALSE, linetype = "dashed", show.legend = TRUE) +

  # --- 1:1 line with legend ---
  geom_abline(aes(intercept = 0, slope = 1, color = "Optimal Line"),
              linetype = "dashed", linewidth = 0.8, show.legend = TRUE) +

  scale_color_manual(
    name = "Lines",
    values = c(
      "Trend Line (Quadratic OLS)" = "blue",
      "Optimal Line" = "red"
    )
  ) +

  # --- Statistics box ---
  annotate(
    "label",
    x = Inf, y = -Inf,
    hjust = 1.1, vjust = -0.5,
    label = paste(
      "RMSE =", round(mean(cv_rmse_quad), 4), "\n",
```

```
      "MAE  =", round(mean(cv_mae_quad), 4), "\n",
      "R²   =", round(mean(cv_r2_quad), 4)
    ),
    size = 4,
    fill = "white"
  ) +

  # --- Labels ---
  labs(
    title = "Actual vs. Predicted Values of NBA 3PMPM w/ Min-0 Predictions",
    subtitle = "(Quadratic OLS 10-Fold CV)",
    x = "Actual Response",
    y = "Predicted Response"
  ) +

  # --- Theme ---
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.25)
  )
```

```
## Warning in geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0), arrow = arrow(type = "closed", : A
## i Please consider using ‘annotate()‘ or provide this layer with data containing
##   a single row.
```

```
## Warning in geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125), arrow = arrow(type = "closed", : A
## i Please consider using ‘annotate()‘ or provide this layer with data containing
##   a single row.
```

```
## ‘geom_smooth()‘ using formula = ’y ~ x’
```




```r
# Build linear + quadratic data
train_lq <- create_linear_quadratic_features(train_data)
test_lq  <- create_linear_quadratic_features(test_data)

# Remove response for regsubsets
x_train <- train_lq[, setdiff(names(train_lq), "response")]
y_train <- train_lq$response

# Best subset selection
best_subset <- regsubsets(
  x = x_train,
  y = y_train,
  nvmax = 10,
  method = "forward"
)

# Extract best model size by BIC (or Cp, adjR2)
best_summary <- summary(best_subset)

best_size <- which.min(best_summary$bic)  # For BIC
# best_size <- which.min(best_summary$cp)    # For Mallow's Cp
# best_size <- which.max(best_summary$adjr2) # For adjusted R²

best_coef <- coef(best_subset, best_size)
selected_vars <- names(best_coef)[-1]  # remove intercept
```

```r
# Fit final model using only selected predictors
formula_final <- as.formula(
  paste("response ~", paste(selected_vars, collapse = " + "))
)

lm_best <- lm(formula_final, data = train_lq)

# Predict on test set
lm_best_pred <- predict(lm_best, newdata = test_lq)
lm_best_pred <- pmax(lm_best_pred, 0)
lm_best_metrics <- calculate_metrics(lm_best_pred, y_test)

model_results <- rbind(
  model_results,
  data.frame(
    Model = "Best Subset (Linear + Quadratic)",
    RMSE = lm_best_metrics[1],
    MAE  = lm_best_metrics[2],
    R2   = lm_best_metrics[3]
  )
)
```

```r
# 2. GNLR GAUSSIAN MODEL FUNCTION
gnlr_gaussian <- function(X, y,
                          max_iter = 50,
                          tol = 1e-6,
                          weight_strategy = c("none", "residual", "density"),
                          kernel_bandwidth = 0.5,
                          lambda = 1e-6) {
  weight_strategy <- match.arg(weight_strategy)
  n <- length(y)

  # Design matrix with intercept
  X_design <- cbind(1, X)
  p <- ncol(X_design)

  # Initial OLS
  lm_init <- lm(y ~ X)
  beta <- coef(lm_init)
  if (length(beta) != p) {
    # In case lm drops columns, fall back to zeros
    beta <- rep(0, p)
  }

  # Helper: density-based weights on fitted values
  density_weights_fun <- function(fitted_vals, bandwidth) {
    # 1D KDE on fitted values
    dens <- density(fitted_vals, bw = bandwidth)
    approx(dens$x, dens$y, xout = fitted_vals, rule = 2)$y
  }

  for (iter in seq_len(max_iter)) {
    beta_old <- beta
```

```
    # Current fitted values
    eta <- as.numeric(X_design %*% beta)
    resid <- y - eta

    # Choose weights
    if (weight_strategy == "none") {
      w <- rep(1, n)
    } else if (weight_strategy == "residual") {
      # Downweight large residuals (robust-ish)
      eps <- 1e-6
      w <- 1 / (abs(resid) + eps)
      w <- w / max(w)
    } else if (weight_strategy == "density") {
      # Upweight dense regions of fitted values
      dens_w <- density_weights_fun(eta, bandwidth = kernel_bandwidth)
      w <- dens_w
      w[w < 0] <- 0
      if (max(w) > 0) w <- w / max(w)
    }

    W <- diag(w, n, n)

    # Weighted least squares update with small ridge for stability
    XWX <- t(X_design) %*% W %*% X_design
    XWy <- t(X_design) %*% W %*% y
    beta <- solve(XWX + diag(lambda, p)) %*% XWy
    beta <- as.numeric(beta)

    # Convergence check
    if (max(abs(beta - beta_old)) < tol) {
      # cat(sprintf("Converged after %d iterations\n", iter))
      break
    }
  }

  # Final fitted values
  eta_final <- as.numeric(X_design %*% beta)
  list(
    coefficients = beta,
    fitted = eta_final,
    residuals = y - eta_final,
    weights = w
  )
}
```

```
# Prepare data matrices
X_train_mat <- as.matrix(train_data[, -ncol(train_data)])
X_test_mat  <- as.matrix(test_data[, -ncol(test_data)])

# Fit GNLR Gaussian with residual-based weighting
gnlr_gauss_fit <- gnlr_gaussian(
  X = X_train_mat,
  y = y_train,
```

```
  weight_strategy = "residual",
  kernel_bandwidth = 0.3
)

# Predictions on test set
X_test_design <- cbind(1, X_test_mat)
gnlr_gauss_pred <- as.numeric(X_test_design %*% gnlr_gauss_fit$coefficients)
gnlr_gauss_pred <- pmax(gnlr_gauss_pred,0)

gnlr_gauss_metrics <- calculate_metrics(gnlr_gauss_pred, y_test)

model_results <- rbind(model_results, data.frame(
  Model = "GNLR Gaussian (residual-weighted)",
  RMSE = gnlr_gauss_metrics[1],
  MAE  = gnlr_gauss_metrics[2],
  R2   = gnlr_gauss_metrics[3]
))
```

```
# Initialize storage
all_actual <- c()
all_predicted <- c()

# Determine actual number of folds
n_folds <- length(cv_folds)

cv_rmse_gnlr <- numeric(n_folds)
cv_mae_gnlr <- numeric(n_folds)
cv_r2_gnlr <- numeric(n_folds)

for(i in 1:n_folds) {
  train_fold <- train_data[-cv_folds[[i]], ]
  valid_fold <- train_data[cv_folds[[i]], ]

  # Prepare matrices for the fold
  X_train_fold <- as.matrix(train_fold[, -ncol(train_fold)])
  y_train_fold <- train_fold$response

  X_valid_fold <- as.matrix(valid_fold[, -ncol(valid_fold)])

  # Fit GNLR Gaussian model
  fold_model <- gnlr_gaussian(
    X = X_train_fold,
    y = y_train_fold,
    weight_strategy = "residual",
    kernel_bandwidth = 0.3
  )

  # Predict on validation fold
  X_valid_design <- cbind(1, X_valid_fold)
  fold_pred <- as.numeric(X_valid_design %*% fold_model$coefficients)
  fold_pred <- pmax(fold_pred, 0)  # Force minimum of zero

  # Store predictions
```

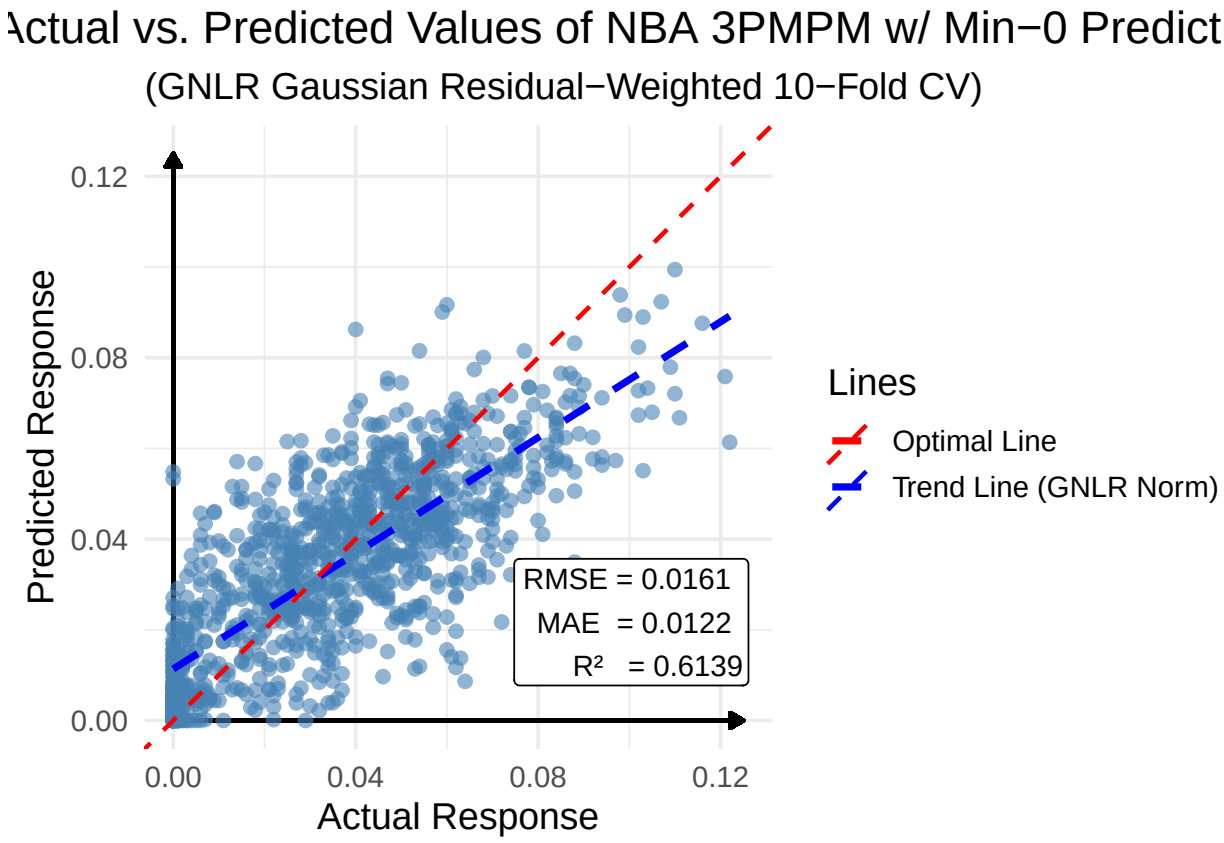
```r
  all_actual <- c(all_actual, valid_fold$response)
  all_predicted <- c(all_predicted, fold_pred)

  # Calculate metrics
  cv_rmse_gnlr[i] <- sqrt(mean((fold_pred - valid_fold$response)^2))
  cv_mae_gnlr[i] <- mean(abs(fold_pred - valid_fold$response))
  cv_r2_gnlr[i] <- 1 - sum((valid_fold$response - fold_pred)^2) /
    sum((valid_fold$response - mean(valid_fold$response))^2)
}

# Create plot data
df_plot <- data.frame(
  Actual = all_actual,
  Predicted = all_predicted
)

ggplot(df_plot, aes(x = Actual, y = Predicted)) +

  # --- Custom axes with arrows ---
  geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +
  geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125),
               arrow = arrow(type = "closed", length = unit(0.2, "cm")),
               linewidth = 0.7, color = "black") +

  # --- Points ---
  geom_point(color = "steelblue", alpha = 0.6) +

  # --- Trend line with legend ---
  geom_smooth(aes(color = "Trend Line (GNLR Norm)"),
            method = "lm", se = FALSE, linetype = "dashed", show.legend = TRUE) +

  # --- 1:1 line with legend ---
  geom_abline(aes(intercept = 0, slope = 1, color = "Optimal Line"),
              linetype = "dashed", linewidth = 0.8, show.legend = TRUE) +

  scale_color_manual(
    name = "Lines",
    values = c(
      "Trend Line (GNLR Norm)" = "blue",
      "Optimal Line" = "red"
    )
  ) +

  # --- Statistics box ---
  annotate(
    "label",
    x = Inf, y = -Inf,
    hjust = 1.1, vjust = -0.5,
    label = paste(
      "RMSE =", round(mean(cv_rmse_gnlr), 4), "\n",
      "MAE  =", round(mean(cv_mae_gnlr), 4), "\n",
```

```
      "R²   =", round(mean(cv_r2_gnlr), 4)
    ),
    size = 4,
    fill = "white"
  ) +

  # --- Labels ---
  labs(
    title = "Actual vs. Predicted Values of NBA 3PMPM w/ Min-0 Predictions",
    subtitle = "(GNLR Gaussian Residual-Weighted 10-Fold CV)",
    x = "Actual Response",
    y = "Predicted Response"
  ) +

  # --- Theme ---
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.22)
  )
```

```
## Warning in geom_segment(aes(x = 0, xend = 0.125, y = 0, yend = 0), arrow = arrow(type = "closed", : A
## i Please consider using ‘annotate()‘ or provide this layer with data containing
##   a single row.
```

```
## Warning in geom_segment(aes(x = 0, xend = 0, y = 0, yend = 0.125), arrow = arrow(type = "closed", : A
## i Please consider using ‘annotate()‘ or provide this layer with data containing
##   a single row.
```

```
## ‘geom_smooth()‘ using formula = ’y ~ x’
```




```
model_results <- rbind(model_results, data.frame(
  Model = "GNLR Gaussian residual-weighted (10-fold CV)",
  RMSE = mean(cv_rmse_gnlr),
  MAE  = mean(cv_mae_gnlr),
  R2   = mean(cv_r2_gnlr)
))
```

```
cat("\n=== MODEL RESULTS ===\n")
```

```
##
## === MODEL RESULTS ===
```

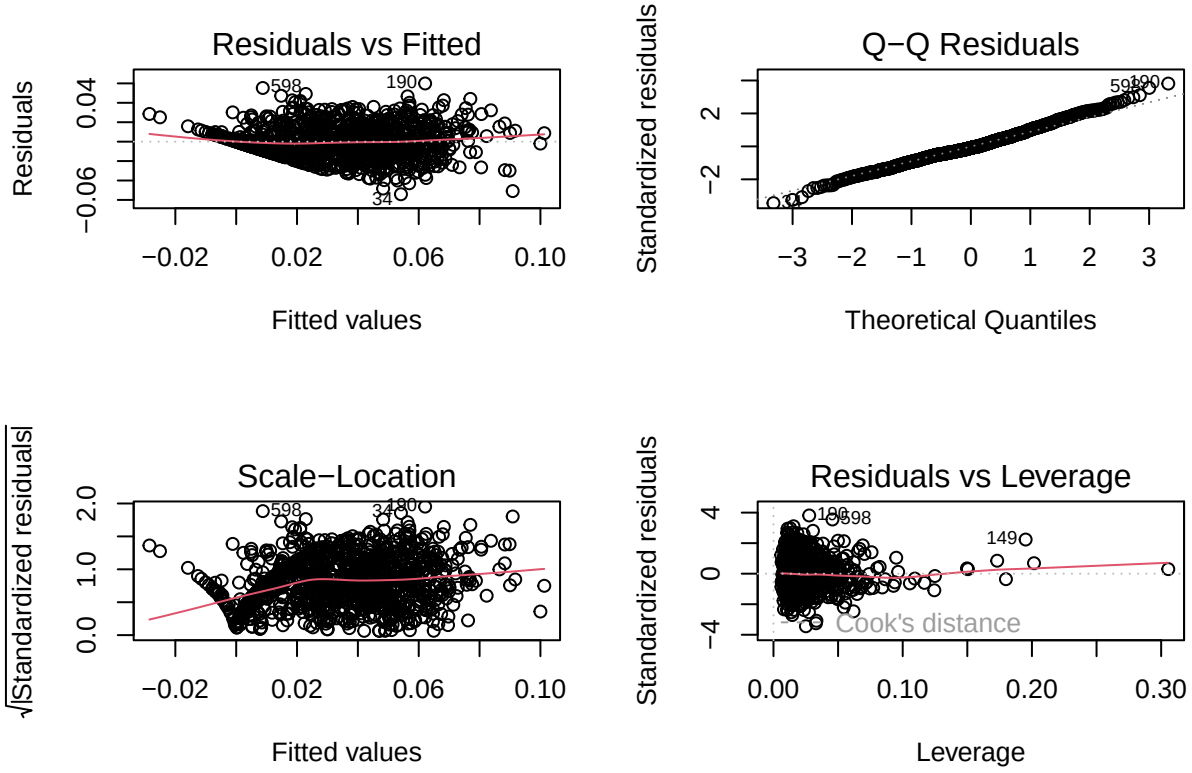
```
print(model_results[order(model_results$RMSE), ])
```

```
##                                                Model       RMSE        MAE
## 4                                     Quadratic OLS 0.01594703 0.01228597
## 3 Linear Regression Min-0 Predictions (10-fold CV) 0.01596340 0.01233445
## 2                   Linear Regression (10-fold CV) 0.01605672 0.01255296
## 7     GNLR Gaussian residual-weighted (10-fold CV) 0.01613951 0.01218288
## 1                            Linear Regression (OLS) 0.01651158 0.01290131
## 6                 GNLR Gaussian (residual-weighted) 0.01666189 0.01258780
## 5                  Best Subset (Linear + Quadratic) 0.01674541 0.01322594
##          R2
## 4 0.6208960
```

```
## 3 0.6230367
## 2 0.6184910
## 7 0.6139099
## 1 0.5935793
## 6 0.5861463
## 5 0.5819865
```

OLS results

```
final_ols <- lm(response ~ ., data = train_data)
par(mfrow = c(2, 2))
plot(final_ols)
```




```
par(mfrow = c(1, 1))

ggplot(data.frame(
  fitted = fitted(final_ols),
  resid = resid(final_ols)
), aes(x = fitted, y = resid)) +
  geom_point(color = "steelblue", alpha = 0.6) +
  geom_hline(yintercept = 0, color = "red", linewidth = 1) +
  labs(
    title = "Fitted vs Residuals",
```

```
    x = "Fitted Values",
    y = "Residuals"
  ) +
  theme_minimal()
```